data. This produces the average rating of a book per bucket. Figure 13.3 shows the expected result from the aggregation execution.


```
{
  "key_as_string" : "2013-01-01T00:00:00.000Z",
  "key" : 1356998400000,
  "doc_count" : 2,
  "avg_rating_per_bucket" : {
    "value" : 4.200000047683716
  }
},
{
  "key_as_string" : "2014-01-01T00:00:00.000Z",
  "key" : 1388534400000,
  "doc_count" : 6,
  "avg_rating_per_bucket" : {
    "value" : 4.383333285649617
  }
}
```


Figure 13.3 Finding the average rating per bucket (sub-aggregation)

The keys are the dates that correspond to the calendar year for documents in that bucket. The notable thing about this query is that an additional object in the bucket, `avg_rating_per_bucket`, consists of the average book rating.

13.3.3 Custom range aggregations

A histogram provides an automatic set of ranges for given intervals. Sometimes we want to segregate data into a range that is not dictated by a strict interval (for example, classifying people into three groups by age: 18 to 21, 22 to 49, and 50+). The standardized interval does not meet this requirement. All we need is a mechanism to customize the ranges—and that's what we use the `range` aggregation for.

The `range` aggregation aggregates documents in a user-defined range. Let's look at it in action by writing a query to fetch books that fall in two categories of ratings: above and below the value 4 (1 to 4 and 4 to 5).

Listing 13.14 Classifying books into two baskets

```
GET books/_search
{
  "size": 0,
  "aggs": {
    "book_ratings_range": {
      "range": {
        "field": "amazon_rating",
        "ranges": [
          {
            "from": 1,
            "to": 4
          },
          {
```

Declares the range aggregation

Applies the aggregation to this field

Sets the customized ranges